# ServiceRegistry

# System Design Description

Eclipse Arrowhead 5.2.2 | core system | Revision 2026-11-09 | REVIEW DRAFT

# Purpose and document structure

This document describes implementation ownership, architecture, persistence, security and operational boundaries. The current cross-language sections below precede the detailed implementation appendices. Appendix dates and release labels belong to the retained baseline, not to a newly qualified release. Existing descriptions have been assembled from the primary trees; no live interoperability or semantic-parity claim is made.

# Java implementation

Primary folder: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry. 1 detailed Java SysDD source document(s) are included in the appendices. Their architecture, operation parameters, models, transport bindings, access rules and errors remain associated with Java. They must not be read as automatically implemented by the ports.

# Go implementation

Primary folder: Eclipse-Arrowhead-go/ah5-core-go/serviceregistry.

## ServiceRegistry - Go 5.2.2

Standalone core system, reorganized on 2026-10-28 following the Eclipse-Arrowhead-java distribution layout. Build and operate from this folder; no original Arrowhead checkout or sibling source module is required. The executable is bin/ServiceRegistry. Entry point: .. Module: arrowhead/serviceregistry. See [DEPLOYMENT.md](DEPLOYMENT.md) for configuration and standalone container use. Existing compatibility limits remain in the copied upstream documentation and [UPSTREAM-README.md](UPSTREAM-README.md). Existing role selection and API behavior are retained. Documentation and license notices are included in this folder. Source provenance and import-only rewrites are listed in the distribution's SOURCE-MANIFEST.json. No running cloud or upstream source was changed.

## Listening port

Default 8443 (server.port in application.properties). Override with server.port, SERVER_PORT=N or -server.port N; the host is server.address. See [LISTENING-PORTS.md](../../LISTENING-PORTS.md) for precedence and all systems.

## Native Go registry design and protocol contract

## Arrowhead 5.2.2 ServiceRegistry (Go)

A Go implementation of the Arrowhead 5.2.2 ServiceRegistry core system, modelled on the Java Spring reference implementation (eclipse-arrowhead/ah5-core-java-spring). HTTP/TLS handling uses the standard library; the optional native monitor uses github.com/rabbitmq/amqp091-go. State is held in memory with an atomic JSON snapshot file. See the parent repository's compatibility report for limitations of the complete Go core-system bundle.

## Build, test, run

go build -o serviceregistry . go test ./... ./serviceregistry -config application.properties Docker: docker build -t ah5-serviceregistry-go . Configuration uses the reference property names (see application.properties); each property can be overridden by an environment variable (SERVER_PORT, DOMAIN_NAME, AUTHENTICATION_POLICY, ...).

## API (base path `/serviceregistry`)

| Area | Endpoints |
|---|---|
| Monitor | GET /monitor/echo |
| Device discovery | POST /device-discovery/register, POST /device-discovery/lookup, DELETE /device-discovery/revoke/{name} |
| System discovery | POST /system-discovery/register, POST /system-discovery/lookup[?verbose], DELETE /system-discovery/revoke |
| Service discovery | POST /service-discovery/register, POST /service-discovery/lookup[?verbose], DELETE /service-discovery/revoke/{instanceId} |
| Management | /mgmt/{devices,systems,service-definitions,service-instances,interface-templates} (POST create, PUT update where applicable, DELETE with names= / serviceInstances=) and POST .../query |
| General management | POST /general/mgmt/logs, GET /general/mgmt/get-config?keys= |

Implemented behaviour: the three authentication policies (declared, certificate, outsourced), the management filter (sysop-only / whitelist / authorization) and the blacklist filter, restricted service discovery (unrestrictedDiscovery metadata), interface policies (open / extendable / restricted) with default templates, property validators, the metadata-requirement DSL, cascading deletes, pagination, Arrowhead error bodies, and self-registration of the ServiceRegistry and its five services at startup (revoked on shutdown; disabled by standalone.mode=true).

## Outsourced authentication with mutual TLS

With authentication.policy=outsourced, server.ssl.enabled=true and server.ssl.client-auth=need every request needs a local-cloud client certificate and an identity token (or the Authentication system's authenticator key):

- The token's system must be the system named by the client certificate CN (token-to-certificate binding), so a stolen

token cannot be used from another certificate.

- A lookup of exactly the identity service is allowed without a token, as in the Java framework: a system must find

Authentication before its first login. The client certificate is still required.

- The registry logs in itself with authenticator.credentials (system name ServiceRegistry), reuses its token for the

identity-verify calls to Authentication and logs in again when the token is refused. Authentication's authenticator key is checked against authenticator.secret.keys.

- Its own services are registered as generic_https with policy CERT_AUTH when client certificates are required.

## sysMonitor (native publisher)

With sysmonitor.enabled=true the registry registers its own sysMonitor 1.0.0 offer (generic_amqps, CERT_AUTH, metadata publisherMode=native) and publishes echo, power and status observations every

sysmonitor.publish.interval seconds (default 30) to the AMQP topic exchange sysmonitor.exchange with routing key <sysmonitor.local.cloud>.sysmonitor.ServiceRegistry.ServiceRegistry.<type> (contract SysMonitor-AMQP-IDD-5.2.2). Messages are persistent and confirmed by the broker. The client (sysmonitor.go, github.com/rabbitmq/amqp091-go) connects with mutual TLS and SASL EXTERNAL, so the broker user is the certificate CN; echo reports degraded when the snapshot file is not writable. The registry also seeds the interface templates generic_amqp and generic_amqps (properties accessAddresses, accessPort, exchange, baseRoutingKey, operations).

| Property | Default | Meaning |
|---|---|---|
| sysmonitor.enabled | false | Register and publish sysMonitor. |
| sysmonitor.broker.address / .port | amqp-broker / 5671 | AMQP broker (TLS). |
| sysmonitor.exchange | arrowhead | AMQP topic exchange. |
| sysmonitor.local.cloud | TestCloud | Local-cloud routing-key word. |
| sysmonitor.publish.interval | 30 | Seconds between publication cycles. |
| sysmonitor.ssl.cert-file, .key-file, .ca-file | - | PEM client certificate, key and CA. |

Environment overrides follow the usual mapping (SYSMONITOR_ENABLED, SYSMONITOR_BROKER_ADDRESS, ...).

## Additional ingresses of the registry's own services

service.registry.extra.interfaces (environment SERVICE_REGISTRY_EXTRA_INTERFACES, default []) is a JSON list of extra interfaces that the registry adds to its own service instances every time it registers itself, for example a private mTLS gateway: Base path and operations are copied from the service's native interface; protocol defaults from the template name and policy to NONE. Because the entries are part of the configuration, they survive registry restarts (the registry revokes its own instances on shutdown). The Java ServiceRegistry of ah5-core-java-spring supports the same property.

## Differences from the Java reference

- No MQTT API.

- JSON-file persistence instead of MySQL.

- Empty metadata is omitted from responses; whole-number floats lose .0 after a persistence round trip.

- Lookup filters evaluate each interface independently (the Java code leaks policy/template/property

flags between interfaces, treated as a bug); the address-type filter is applied independently of property requirements; mandatory template properties are checked uniformly.

- addressType in management queries is case-insensitive.

- String-set properties keep input order with duplicates removed.

- Outbound calls to ConsumerAuthorization and Blacklist services are best-effort implementations of

the documented verify/check operations and have not been tested against the real services. Identity login and verification are verified in the Arrowhead test cloud (Arrowhead-test-cloud-go).

- Tested with unit/httptest integration tests and in the Go test cloud.

- The parent Go core bundle additionally tests a temporary native registry process with the new

Go orchestration and authorization implementations. Management authorization now sends the v5.2.2 targetType / target / scope fields and registered operation names; outsourced tokens grant sysop access only when Authentication returns sysop=true, not merely for a Sysop name.

### Detailed implementation evidence

0 native Go SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

## Rust implementation

Primary folder: Eclipse-Arrowhead-rust/ah5-core-rust/serviceregistry.

### ServiceRegistry (Rust)

Arrowhead 5.2.2 core system. src/main.rs is built as binary serviceregistry by the arrowhead-core-rust crate in ... serviceregistry.json and documentation/ link to ../examples and ../documentation/ServiceRegistry.

### Detailed implementation evidence

1 native Rust SysDD document(s) are included in the appendices. Their protocol and storage descriptions apply to that implementation only. Current primary-folder ownership and current listening-port notes above supersede historical paths or example ports in those retained documents.

### Build and state ownership

Core entry points are built by arrowhead-core-rust; the ordinary support entry points are built by arrowhead-support-rust using the shared support runtime. LocalCloudManager has its own crate. Per-system documentation links may point into the shared documentation catalog. Independent identity and state configuration is still required. Read the native appendix for persistence and restart semantics.

## Cross-language qualification

Identical system names or HTTP paths do not establish interoperability. For the selected implementation, verify registered service versions and metadata, method/path and payload models, null and optional-field handling, success/error bodies, authentication and per-operation authorization, timeout/retry behavior and persistence across restart. Java MQTT/AMQP descriptions apply to a port only where its own implementation evidence explicitly supports them. Missing transports must not be advertised. State conversion from Java SQL to native snapshots, secret provisioning, backup/restore and rollback require implementation-specific procedures. No data migration, deployment or cloud reconfiguration was performed for this document set.

## Detailed appendices and provenance

- Java: Eclipse-Arrowhead-java/ah5-core-java-spring/serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysDD.tex

- Rust: Eclipse-Arrowhead-rust/ah5-core-rust/documentation/ServiceRegistry/ServiceRegistry-SysDD.tex

The package sources/ folder retains the selected LaTeX, figures, bibliography, language README/build evidence and source snapshots for newly described contracts. source-manifest.json records SHA-256 hashes of original inputs. build-report.json records whether an appendix used an existing newer PDF or was compiled from source.

# ServiceRegistry

## System Design Description

**Abstract**
This document describes the design and implementation of the ServiceRegistry core system of Arrowhead 5.2.2 in Java Spring (`ah5-core-java-spring/serviceregistry`).

# Contents

**1 Overview** **3**

**2 Implementation** **4**
2.1 Implementation language and tools . . . . . 4
2.2 Functional properties implementation . . . . . 4
2.3 Non functional properties implementation . . . . . 5
2.4 Deployment . . . . . 5
2.5 Local modifications . . . . . 6
2.6 Verification . . . . . 6

**3 Services** **7**

**4 References** **7**

**5 Revision History** **9**
5.1 Amendments . . . . . 9
5.2 Quality Assurance . . . . . 9

# 1 Overview

This document describes the ServiceRegistry system implementation, which realizes the ServiceRegistry SysD [1]. The ServiceRegistry is the first core system of an Arrowhead local cloud. It keeps the authoritative register of the devices, systems and service instances of the cloud and of the interface templates that describe how services are reached. Every other system registers itself and its services there and finds the core services through it; orchestrators use it to discover candidate providers. In Section 2, we describe implementation details of the system.

# 2 Implementation

This implementation is based on the SysD document ServiceRegistry [1], the upstream Eclipse Arrowhead Java Spring implementation 5.2.2 [2] and the local changes listed in Section 2.5.

## 2.1 Implementation language and tools

- Java 21, Spring Boot 3.4.9 (web, security, data JPA, Quartz), springdoc-openapi 2.8 (Swagger UI), Log4j 2.
- Maven module `serviceregistry` of `ah5-core-java-spring`; it depends on `arrowhead-common-utils` and `arrowhead-data-transfer-objects` 5.2.2 from the local `ah5-common-java-spring` (Eclipse Paho MQTT v3 1.2.5, RabbitMQ AMQP client 5.22.0).
- Database: MariaDB/MySQL (MariaDB Connector/J; MySQL 8.4 server in the test cloud), schema validated by Hibernate (`ddl-auto=validate`); SQL scripts in `src/main/resources/database`.
- Stateless request processing; all state is in the database. Scheduled jobs use Quartz.

## 2.2 Functional properties implementation

The Java packages below `eu.arrowhead` implement the system:

| Package | Responsibility |
|---|---|
| `api.http, api.mqtt` | REST controllers and MQTT topic handlers of the five services; the lookup filter that marks restricted lookups. |
| `service` | Device, system and service discovery and management logic; `ReservedServiceDefinitions` (J9); discovery and interface policies. |
| `service.validation, service.normalization` | Input validation and name normalization; interface validators per template property. |
| `jpa` | Entities and repositories of the register; database services with bulk operations. |
| `init` | Registration of the system's own services at start-up, extra interfaces and interface policy override (local). |
| `collector` | `DatabaseCollectorDriver`: the ServiceRegistry resolves services from its own database. |

Every request passes the common filter chain: authentication filter (by `authentication.policy`), optional blacklist filter, management service filter (for management services), then the controller, validation and normalization, the service logic and the database service. The same logic is reachable through the MQTT topic handlers.

### 2.2.1 Data handled and stored

Database `ah_serviceregistry`:

| Tables | Content |
|---|---|
| `device, device_address` | Devices and their addresses. |
| `system_, system_address, device_system_connector` | Systems, addresses and device assignment. |
| `service_definition` | Service definition names. |
| `service_instance, service_instance_interface` | Service instances and their interfaces (properties as JSON). |

| Tables | Content |
|---|---|
| `service_interface_template`, `service_interface_template_property` | Interface templates and property rules; default inserts add `generic_http(s)`, `generic_mqtt(s)` and, locally, `generic_amqp(s)`. |
| `logs` | Log entries for `get-log`. |

The data models of the interfaces are the Java records of `eu.arrowhead.dto` (common library); the IDD documents name them without the `DTO` suffix. The records used by this system are: `AddressDTO`, `DeviceListRequestDTO`, `DeviceListResponseDTO`, `DeviceLookupRequestDTO`, `DeviceQueryRequestDTO`, `DeviceRequestDTO`, `DeviceResponseDTO`, `KeyValuesDTO`, `LogEntryDTO`, `LogEntryListResponseDTO`, `LogRequestDTO`, `MetadataRequirementDTO`, `PageDTO`, `ServiceDefinitionListRequestDTO`, `ServiceDefinitionListResponseDTO`, `ServiceDefinitionResponseDTO`, `ServiceInstanceCreateListRequestDTO`, `ServiceInstanceCreateRequestDTO`, `ServiceInstanceInterfaceRequestDTO`, `ServiceInstanceInterfaceResponseDTO`, `ServiceInstanceListResponseDTO`, `ServiceInstanceLookupRequestDTO`, `ServiceInstanceQueryRequestDTO`, `ServiceInstanceRequestDTO`, `ServiceInstanceResponseDTO`, `ServiceInstanceUpdateListRequestDTO`, `ServiceInstanceUpdateRequestDTO`, `ServiceInterfaceTemplateListRequestDTO`, `ServiceInterfaceTemplateListResponseDTO`, `ServiceInterfaceTemplatePropertyDTO`, `ServiceInterfaceTemplateQueryRequestDTO`, `ServiceInterfaceTemplateRequestDTO`, `ServiceInterfaceTemplateResponseDTO`, `SystemListRequestDTO`, `SystemListResponseDTO`, `SystemLookupRequestDTO`, `SystemQueryRequestDTO`, `SystemRegisterRequestDTO`, `SystemRequestDTO`, `SystemResponseDTO`.

## 2.3 Non functional properties implementation

### 2.3.1 Security

TLS and client authentication are configured with the Spring `server.ssl.*` properties (PKCS#12 keystore and truststore). The authentication filters of the common library implement the four authentication policies; `TokenCertificateBinding` binds identity tokens to the client certificate and `CrlRevocationConfig` enables CRL checking in Tomcat (both local). The private key of the keystore is also used to sign the system's outgoing requests where required.

### 2.3.2 Power management

None; the system is a server process without power states. The `sysMonitor` `power` observation always reports line supply.

### 2.3.3 Internal monitoring

Log4j 2 logs to stdout and rolling files; log entries are also written to the `logs` table and served by `generalManagement.get-log`. The native `sysMonitor` publisher (`eu.arrowhead.common.sysmonitor`) publishes `echo`, `power` and `status` every 30 s.

### 2.3.4 Configuration

Configuration is read from `application.properties` and environment variables (Spring relaxed binding, e.g. `SERVICE_DISCOVERY_POLICY`); secrets come from files through `SPRING_CONFIG_IMPORT=configtree:/run/secrets/`. The properties are listed in the SysD. `generalManagement.get-config` returns the public ones.

## 2.4 Deployment

The test cloud (`Arrowhead-test-cloud-java`) builds the image `arrowhead-serviceregistry:5.2.2-sysmonitor` from the released image `aitiaiiot/arrowhead-serviceregistry:5.2.2` with the locally

built `arrowhead-serviceregistry-5.2.2.jar` (`java-patch/Dockerfile`). The container runs read-only with the configuration baked into `/app/config`, only its keystores mounted, and publishes port 8443 on the loopback address of the host. Its database runs in a separate container.

## 2.5 Local modifications

Compared to upstream 5.2.2, this workspace changes:

- Startup is refused when the effective configuration is below the secure profile (`SecurityProfileGuard`).
- Reserved service definitions for the core and support systems (finding J9), configurable with `service.definition.reserved`.
- The own interfaces are registered with `CERT_AUTH` behind TLS and with `NONE` for declared identities or without TLS; `service.interface.policy` overrides this; `service.registry.extra.interfaces` registers additional ingress interfaces.
- Native `sysMonitor` publisher (common library), always registered with `CERT_AUTH`; interface templates `generic_amqp` and `generic_amqps` in the default inserts.
- Token-to-certificate binding of identity tokens and CRL checking of client certificates (common library).
- MariaDB Connector/J instead of MySQL Connector/J in the default configuration.

## 2.6 Verification

The module has 31 JUnit test classes; the regression report set recorded on 2026-10-20 covers 699 tests without failures. Live-cloud smoke tests are a separate verification step; the plant-management migration is not claimed as deployed by this update.

# 3 Services

The implementation of the services is based on the SD and IDD documents referenced below.

| **Services produced** | **SysD ref** | **SD ref** | **IDD ref** |
|---|---|---|---|
| `deviceDiscovery` | [1] | [3] | [4], [5] |
| `systemDiscovery` | [1] | [6] | [7], [8] |
| `serviceDiscovery` | [1] | [9] | [10], [11] |
| `serviceRegistryManagement` | [1] | [12] | [13], [14] |
| `generalManagement` | [1] | [15] | [16], [17] |
| `sysMonitor` | [1] | [18] | [19] |
| | | | |
| **Services consumed** | **SysD ref** | **SD ref** | **IDD ref** |
| identity | [20] | [21] | – |
| authorization | [22] | [23] | – |
| blacklistDiscovery | Blacklist | – | – |
| sysMonitor (publisher) | AMQP broker (SysMonitor) | – | – |

# 4 References

[1] J. Delsing, “ServiceRegistry – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, serviceregistry/documentation/v5.2.2/ServiceRegistry-v5.2.2-SysD.pdf.

[2] AITIA International, “Eclipse arrowhead 5 core systems (java spring) and documentation,” https://aitia-iiot.github.io/ah5-docs-java-spring/, 2025.

[3] J. Delsing, “deviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-deviceDiscovery-SD.pdf.

[4] ——, “deviceDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-deviceDiscovery-HTTP-IDD.pdf.

[5] ——, “deviceDiscovery MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-deviceDiscovery-MQTT-IDD.pdf.

[6] ——, “systemDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-systemDiscovery-SD.pdf.

[7] ——, “systemDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-systemDiscovery-HTTP-IDD.pdf.

[8] ——, “systemDiscovery MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-systemDiscovery-MQTT-IDD.pdf.

[9] ——, “serviceDiscovery – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceDiscovery-SD.pdf.

[10] ——, “serviceDiscovery HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceDiscovery-HTTP-IDD.pdf.

[11] ——, “serviceDiscovery MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceDiscovery-MQTT-IDD.pdf.

[12] ——, “serviceRegistryManagement – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceRegistryManagement-SD.pdf.

[13] ——, “serviceRegistryManagement HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceRegistryManagement-HTTP-IDD.pdf.

[14] ——, “serviceRegistryManagement MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-serviceRegistryManagement-MQTT-IDD.pdf.

[15] ——, “generalManagement – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-generalManagement-SD.pdf.

[16] ——, “generalManagement HTTP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-generalManagement-HTTP-IDD.pdf.

[17] ——, “generalManagement MQTT/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-generalManagement-MQTT-IDD.pdf.

[18] ——, “sysMonitor – Service Description (ServiceRegistry),” Eclipse Arrowhead, SD 5.2.2, 2026, serviceRegistry-v5.2.2-sysMonitor-SD.pdf.

[19] ——, “sysMonitor AMQP/TLS/JSON – Interface Design Description (ServiceRegistry),” Eclipse Arrowhead, IDD 5.2.2, 2026, serviceRegistry-v5.2.2-sysMonitor-AMQP-IDD.pdf.

[20] ——, “Authentication – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, authentication/documentation/v5.2.2/Authentication-v5.2.2-SysD.pdf.

[21] ——, “identity – Service Description (Authentication),” Eclipse Arrowhead, SD 5.2.2, 2026, authentication-v5.2.2-identity-SD.pdf.

[22] ——, “ConsumerAuthorization – System Description,” Eclipse Arrowhead, SysD 5.2.2, 2026, consumer-authorization/documentation/v5.2.2/ConsumerAuthorization-v5.2.2-SysD.pdf.

[23] ——, “authorization – Service Description (ConsumerAuthorization),” Eclipse Arrowhead, SD 5.2.2, 2026, consumerAuthorization-v5.2.2-authorization-SD.pdf.

# 5 Revision History

## 5.1 Amendments

| No. | Date | Version | Subject of Amendments | Author |
|---|---|---|---|---|
| 1 | 2026-11-07 | 5.2.2 | Document created from the Arrowhead LaTeX templates for the 5.2.2 Java Spring implementation, including the local extensions of this workspace. | Jerker Delsing |

## 5.2 Quality Assurance

| No. | Date | Version | Approved by |
|---|---|---|---|
| 1 | | 5.2.2 | |

# 1 ServiceRegistry

## 1.1 System overview

Native Rust implementation of the local Java module `serviceregistry/`. Each named executable checks its configured role and owns an independent identity, listen port and state file. Shared Rust modules implement the runtime; no Java, Python or Go process implements the service.

## 1.2 Produced and consumed services

Produced HTTPS definitions: `deviceDiscovery`, `echo`, `generalManagement`, `serviceDiscovery`, `serviceRegistryManagement`, `systemDiscovery`. Service versions are 1.0.0; the target system version is 5.2.2. Authorities consumed as applicable are Authentication, ServiceRegistry and ConsumerAuthorization. Explicit pinned HTTPS origins and valid cloud identities must be provisioned.

## 1.3 Architecture and persistent state

The listener uses Tokio, Hyper/Axum and rustls with mandatory client certificates. Verified peer identity is supplied through a private request extension, never a declared-identity header. Business modules share locked JSON state with clone/validate/fsync/rename transactions. Credentials and keys are separate owner-only files. Cross-system operations and callbacks are not distributed transactions.

## 1.4 Security

Local cloud CN/profile, certificate chain and validity are verified. Identity tokens are checked against Authentication and bound to the client certificate. Management requires a verified sysop or explicit broad whitelist. Outbound peer chain/SAN and expected system CN/profile are checked before credentials are sent. Optional PEM CRLs are enforced at startup-configured trust boundaries and require process restart to reload. No plaintext HTTP listener, TLS bypass or declared SYSTEM identity mode is provided.

## 1.5 Implementation status and delimitations

This is a development profile, not a certified interchangeable Java replacement. HTTP route presence is not semantic equivalence. Java database migration, MQTT application APIs and production qualification remain outstanding. Registry records, interface templates, expiry, cascades, authorized discovery and management are implemented. Java SQL import, the full property-validator catalogue, normalization, advanced version/-time/address matching and all discovery-policy options need additional work. Unsupported custom validators fail explicitly; they are not silently accepted.

# 2 Implementation structure

See `src/security.rs`, `src/store.rs`, `src/web.rs` and the role-specific module. Transactions clone state and publish only after durable rename. State locking prevents concurrent processes from sharing the same path. A directory-sync failure after rename is an uncertain durable commit, not an automatic rollback. HTTPS push callbacks are at-least-once across crash recovery; downstream effects can require reconciliation.

## 2.1 HTTPS interface operations

HTTPS uses JSON unless an operation specifies text. Request bodies are limited to 1 MiB; upstream responses to 8 MiB. Identity-token format: `Bearer IDENTITY-TOKEN//token`. Certificate-only identity operations still require verified mutual TLS and apply credential/certificate binding.

| Method | Path | Access |
|---|---|---|
| GET | /serviceregistry/general/mgmt/get-config | management |
| POST | /serviceregistry/general/mgmt/logs | management |
| GET | /serviceregistry/monitor/echo | certificate |
| POST | /serviceregistry/device-discovery/register | identity |
| POST | /serviceregistry/device-discovery/lookup | identity |
| DELETE | /serviceregistry/device-discovery/revoke/{name} | identity |
| POST | /serviceregistry/system-discovery/register | identity |
| POST | /serviceregistry/system-discovery/lookup | identity |
| DELETE | /serviceregistry/system-discovery/revoke | identity |
| POST | /serviceregistry/service-discovery/register | identity |
| POST | /serviceregistry/service-discovery/lookup | identity |
| DELETE | /serviceregistry/service-discovery/revoke/{instanceId} | identity |
| POST | /serviceregistry/mgmt/devices/query | management |
| POST | /serviceregistry/mgmt/devices | management |
| DELETE | /serviceregistry/mgmt/devices | management |
| PUT | /serviceregistry/mgmt/devices | management |
| POST | /serviceregistry/mgmt/systems/query | management |
| POST | /serviceregistry/mgmt/systems | management |
| DELETE | /serviceregistry/mgmt/systems | management |
| PUT | /serviceregistry/mgmt/systems | management |
| POST | /serviceregistry/mgmt/service-definitions/query | management |
| POST | /serviceregistry/mgmt/service-definitions | management |
| DELETE | /serviceregistry/mgmt/service-definitions | management |
| POST | /serviceregistry/mgmt/service-instances/query | management |
| POST | /serviceregistry/mgmt/service-instances | management |
| DELETE | /serviceregistry/mgmt/service-instances | management |
| PUT | /serviceregistry/mgmt/service-instances | management |
| POST | /serviceregistry/mgmt/interface-templates/query | management |
| POST | /serviceregistry/mgmt/interface-templates | management |
| DELETE | /serviceregistry/mgmt/interface-templates | management |

Errors contain errorMessage, errorCode, exceptionType, origin and timestamp. Exact validation/error parity and all Java security-policy settings remain release gates. Unknown routes return 404; unavailable authorities fail closed.